Architectural Digital Twin cum Simulator

Execution plan v3 · decision-of-record · built from the Ideate Phase PDF · web app on one laptop · full scope, built in stages

1. What it is

A browser app where an architecture student builds a parametric one or two-storey building (floors, bays, roof type, walls, doors, windows), changes its structure (material, beam sizes, loads) and sees calculated results: which beams pass or fail, how much each bends, and the total weight and cost. Students compare materials and compare saved design versions. It is a learning tool: the Ideate PDF asks for a student-centred experience that makes design consequences easier to understand, not a professional engineering twin.

2. Decisions

Tag	Decision	Why
LOCKED	Name stays "Architectural Digital Twin cum Simulator". The app states it is a simplified model with no live-data link.	Matches the PDF. Honest if a teacher asks what makes it a "twin".
LOCKED	Frontend: Three.js + plain JavaScript, no build step. three.module.min.js and OrbitControls.js are copied into frontend/vendor and loaded with an import map.	Runs offline. Nothing to compile.
LOCKED	Backend: Python FastAPI + SQLite (backend/adt.db). The same server serves the frontend.	A real backend with a database you can show. One command.
LOCKED	All calculation is in frontend/calc.mjs (browser). Backend stores and returns data only.	Instant slider feedback. One place to test the maths.
LOCKED	Building generator (parametric): 1 or 2 floors; 1 to 3 bays in each direction; bay span 2 to 8 m; floor height 2.5 to 4 m; roof type flat, shed or gable; exterior walls with door and window counts per side; optional interior partition walls; toggles for walls, roof and each floor.	PDF asks for a modifiable building model with selectable components. Richer than one room, still boxes and prisms.
LOCKED	Structure per floor: columns on every grid intersection, beams along both directions between columns, slab on top. Every part is clickable and shows size, material, weight and cost.	PDF idea 6: component information.
LOCKED	Look and feel follows the PDF images (docs/reference/): warm cream background, teal headings, numbered round badges, rounded cards, a dashboard of panels around the 3D view. We match the layout, colours and panels, not the photo-realistic textures of the concept images.	You asked for it to look like the PDF. Realistic renders are not possible with simple 3D boxes.
LOCKED	Screen layout (from the mind map and idea 5 image): left Design Parameters panel; centre 3D view; right Expected Results panel with arrows showing what changed; bottom cards for material comparison and "About this result".	Matches the Interactive Building Model, Visual Comparison and Cause and Effect nodes.
LOCKED	Original vs Modified: two 3D views side by side with highlighted differences and a layer toggle (Structure, Architecture, Materials). Daylight layer is LATER.	Mind map "Visual Comparison"; idea 5 image.
LOCKED	Deformation colour map: beams coloured on a blue to red scale by deflection as a share of the limit, with a legend. Pass and fail is also written as text.	Idea 1 image shows a colour-coded structure.
LOCKED	Cause and Effect panel: after a change it lists what moved, with up and down arrows and numbers: deflection, pass count, weight, cost, and a daylight indicator (window area divided by floor area, no ray tracing). Heat gain is LATER.	Mind map "Cause & Effect Learning". Only effects we can calculate honestly are shown.
LOCKED	"About this result" panel: data source, assumptions, limitations and when to use professional analysis.	Mind map "Limitations & Accuracy Panel". Replaces the plain limits note.
LOCKED	Parameters beyond the beam: roof type and roof pitch, window size and count per side, room (bay) size. Roof pitch changes roof area, weight and cost; window size changes the daylight indicator. They do not change the beam load unless the user changes the load slider.	Mind map lists roof slope, window position and room size. Keeps the physics simple and honest.
LOCKED	Material card: swatches for timber, steel and concrete with density, thermal conductivity, durability (Low, Medium or High, an assumed rating) and cost estimate, side by side.	Idea 2 image and PDF idea 2 text.

Tag	Decision	Why
LOCKED	Simulation output: beam deflection (simply supported and cantilever, even load, limit span/250). Analyse one selected beam, or check all beams at once: each beam is coloured green (pass) or red (fail) in the 3D view.	PDF ideas 1 and 5. Whole-building check gives the design-consequence feel.
LOCKED	Totals for the whole building: weight and cost estimate from component volumes and material data. Material comparison for the whole structure and for one beam.	PDF idea 2.
LOCKED	Input is sliders and typed values plus 4 ready-made design alternatives to compare, as the PDF describes: Small house (1 floor), Two-storey house, Hall, Shed. Each differs in layout, roof and materials. No blueprint upload.	Blueprint reading is unreliable and not in the PDF.
LOCKED	Saved designs store all parameters as JSON. Compare any two versions side by side: parameters, results, pass or fail counts, total weight and cost.	PDF idea 7.
DROPPED	Login, accounts, blueprint or 3D-file upload, rainwater flow (idea 4 is not in the mind map).	Not in the selected direction.
LATER	1) Full sunlight and heat simulation (idea 3, heat map as in its image) and the Daylight layer: windows are already placed. Section view. 2) Guided learning challenges (idea 8). 3) Student test results feed the next design round.	After the core is tested with students.

3. Calculation (frontend/calc.mjs)

I = b·h³/12. Self weight = density·9.81·b·h/1000 kN/m. w = applied load + self weight. Simply supported: δ = 5wL⁴/(384EI). Cantilever: δ = wL⁴/(8EI). Pass if δ ≤ L/250. Beam length is the bay span. Code works in metres, newtons and pascals and shows mm and kN/m. Material numbers are rounded teaching assumptions and are labelled so in the app. Each floor slab is a concrete slab of 120 mm; columns are 300×300 mm; walls are 200 mm thick; these are fixed assumptions shown in the limits text.

Test	Material	Span	Section b×h	Load	Type	Expected δ
A	Concrete	5 m	250×350 mm	10 kN/m	Simply supported	4.40 mm
B	Steel	4 m	100×200 mm	5 kN/m	Simply supported	1.64 mm
C	Timber	2 m	100×200 mm	2 kN/m	Cantilever	5.78 mm

Count tests the generator must pass (f floors, bays bx and bz): columns = f·(bx+1)(bz+1); beams = f·[bx(bz+1) + bz(bx+1)]; slabs = f. Total weight equals the sum of every component weight. Trend tests: more span or load gives more deflection; more depth or stiffer material gives less. Materials (E, density, heat conductivity, cost per m³): timber 11 GPa, 600 kg/m³, 0.13 W/mK, ₹30,000. Steel 200 GPa, 7850 kg/m³, 50 W/mK, ₹470,000. Concrete 25 GPa, 2400 kg/m³, 1.4 W/mK, ₹20,000. Costs are rough assumptions (open item O6).

4. Structure, API and set-up

Item	Detail
Folders	backend/ (main.py, db.py, seed.py, tests/test_api.py) · frontend/ (index.html, app.js, calc.mjs, building.mjs, vendor/, tests/*.test.mjs) · prototype/ · docs/ · run.sh · run.bat · README.md
Run	./run.sh (Mac/Linux) or run.bat (Windows) creates the database if missing, seeds materials, starts http://localhost:8000. API docs at /docs.
Tests	Backend: python -m pytest. Frontend logic: node --test frontend/tests. Both must pass before every commit.
materials	id, name, e_gpa, density, conductivity, cost_per_m3, note
designs	id, name, created_at, params_json (every slider and choice), summary_json (pass and fail counts, worst deflection, total weight, total cost)
Endpoints	GET /health · GET /materials · GET /designs · POST /designs (validates ranges, known material, beam type, roof type) · GET /designs/{id} · DELETE /designs/{id}
Pre-flight	Python 3.10+; pip install fastapi uvicorn pytest httpx. Node 18+. Chrome. In a temp folder run npm i three, then copy build/three.module.min.js and examples/jsm/controls/OrbitControls.js to frontend/vendor/. Claude Code logged in. Unzip the kit, git init. Open prototype/digital-twin-simulator.html once: it is the single-beam fallback demo.

5. Stages and gates

One stage per CLI session, in order. A stage is done only when its one-sentence gate passes with real test output, and it is committed to git. No deadlines: quality first. Milestones M1 and M2 mark points where the app can be demonstrated.

#	Stage	Gate (one sentence)
0	Setup: folders, git, run scripts, confirm prototype opens	Python, Node and the prototype all run on this laptop.
1	Backend and materials: FastAPI, SQLite, seed, designs CRUD, serves frontend, pytest	pytest passes, /docs opens, /materials returns timber, steel and concrete.
2	Calculation: calc.mjs, both beam types, whole-building check, totals	Tests A, B, C pass within 0.01 mm and the trend tests pass.
3	Building generator: building.mjs makes floors, bays, columns, beams, slabs, walls, door, windows, roof from parameters	Component counts match the formulas for 5 different parameter sets.
4	Viewer and look: 3D scene from the generator, left parameters panel, layer toggles, click for component details, deformation colour map with legend, PDF-style theme	Changing floors, bays, roof, windows or material updates the model, clicking any part shows size, material, weight and cost that match a hand calculation, and the screen layout matches the PDF mind map.
	M1: CHECKPOINT 1 (code, math, spec, design reviewers)	Core demo works end to end without the version feature.
5	Analysis panel: select a beam, analyse it, check all beams, totals, material card	Check-all flags exactly the beams the calculation fails, and totals equal the sum of the parts.
6	Versions and comparison: save and load through the API, original vs modified in two 3D views with highlighted differences, Cause and Effect panel	Restart the server and a saved design is still there; the modified design shows side by side with the original and the panel lists every changed value with an arrow.
7	Four design alternatives, plain-language messages, the "About this result" panel	Every result opens an About panel with source, assumptions and limits, and each design alternative loads without error.
8	Polish, README, clean-folder test, git tag v1	From a clean copy one command starts the app and the demo script runs end to end.
	M2: CHECKPOINT 2 (all five reviewers)	No open problems. Ready to demo.
9	Student test (people, not agents): 3 to 5 students	Each finishes one task (find a design where all beams pass) without help.

6. Risks and how the plan handles them

Risk	Control
Generator bugs make wrong models	Count formulas are tests, and the math-reviewer re-derives them separately.
3D becomes slow with many parts	Cap at 2 floors and 3×3 bays (at most 48 beams). Reuse geometry. The code-reviewer checks frame rate on a normal laptop.
Scope grows	spec-reviewer rejects anything not in the plan. New ideas go to LATER.
Something breaks late	Git commit at every gate; roll back to the last good commit. The prototype stays as the fallback demo.
Maths looks right but is wrong	Three hand-checked cases plus trend tests, run by an independent script.
Credits run out	One stage per session, short replies, reviewers only at the two checkpoints.

7. Reviewer agents (read-only, in .claude/agents)

Agent	Checks	Runs at
code-reviewer	Runs tests and the app, tries bad input (negative size, unknown id, out-of-range floors), clean checkout works, frame rate acceptable.	M1 and M2
math-reviewer	Recomputes formulas, units, component counts and totals in a separate script; tests A, B, C and trends.	M1 and M2
spec-reviewer	Work matches this plan and the Ideate PDF; no extra features; each gate really passes.	M1 and M2

Agent	Checks	Runs at
design-reviewer	Layout, contrast, spacing, consistency, narrow screens, 3D view not cramped, pass and fail not shown by colour alone, original vs modified clear.	M1 and M2
student-reviewer	Reads the screen as a first-year student: jargon, missing assumptions, unclear messages.	M2

They run in parallel. The main agent fixes what they report, then re-runs only the reviewer that failed.

8. Demo script (3 minutes) and likely questions

Step	Show
1	Pick the Two-storey house design, rotate it, hide the walls to show the frame, click a column and a window.
2	Press check-all: some beams turn red. Click one, see how far it bends, make it deeper or switch to steel until it turns green.
3	Open the material comparison: weight, cost and pass count for timber, steel and concrete.
4	Save the original and the modified design, show them side by side. Restart the server: they are still there.
5	Open /docs, show the database rows and the passing tests.

Question	Answer
What makes it a digital twin?	It is a virtual model that responds to changes in the real design parameters. It has no live sensor link, and the app says so. Linking real data is future work.
Why a backend?	It stores the material data and every saved design so versions can be reopened and compared.
How accurate is it?	It uses the standard beam formula with stated assumptions (solid rectangle, even load, simple supports). It is for learning, not for designing a real building.
Where do material values come from?	Rounded teaching values. Citing a source is open item O6.
What next?	Student test, then sunlight and heat exposure, then guided challenges.

9. Sign-off appendix

ID	Item	Status
O1	Which simulation outputs count as "supported" (PDF does not say)	LOCKED: beam deflection, whole-building check, totals. Change if the teacher expects more.
O2	"Digital twin" wording with no live data link	LOCKED: keep the name, state the limit in the app.
O3	Backend calculates or browser calculates	LOCKED: browser. Backend stores data.
O4	Blueprint or 3D-file upload	DROPPED. Revisit after the student test.
O5	Sunlight, then learning challenges	LATER.
O6	Material values and costs are assumptions	OPEN: replace with a cited source before final submission.
O7	Operating system (decides run.sh or run.bat)	OPEN: tell the CLI in the first message.